A TUTORIAL FOR NEW ENGINEERS

How to Think About Systems

We start with one engineer's Tuesday, and we finish by designing Instagram.

What we do, and where it ends up.

PART ZERO

Before anything is a system, it is a Tuesday.

THE RULES OF THIS PART

Maya has been an engineer for seven months, and today she wants one thing.

She wants pull request 482 merged before the release window closes at four o'clock.

Watch her Tuesday without naming anything. There is no vocabulary in this part on purpose — every word we define in Part one happens in the slides that follow, unlabeled, and we come back afterwards to name each one.

MORNING

The day is already pushing back before Maya reaches the office.

06:55

The shower

Too hot. She turns it down. Too cold. She overshoots twice before it settles.

08:05

The train map

Wildly wrong about geography. She has never once been lost using it.

08:20

A signal failure

Nine minutes, stopped, and nothing anyone aboard can do. She opens Instagram without deciding to.

08:50

At her desk

Wifi, the VPN, the package registry. She notices none of them.

09:05

The registry goes down

Three teams cannot build. Now she notices.

09:15 · STANDUP

Four people, and it matters enormously which way they are wired.

The same four stand in a circle for ten minutes. On Monday they each reported upward to the manager and the meeting took twenty minutes and settled nothing. Today they talk across to each other, and Maya learns in one sentence that Priya has already read the code she was about to start on.

Same four people, both days.

Get 482 merged before the window closes.

MAYA'S STICKY NOTE, 09:30. IT IS THE ONLY THING ON IT.

09:45 · IN THE WAY

Four things bound Maya's day, and none of them is her.

The build takes twelve minutes

Every push costs twelve minutes she cannot compress, argue with, or skip.

The team is three people

Nobody is free to take the review off her hands this morning.

The one reviewer is asleep

He is six time zones away and will not read anything before three o'clock.

Production data stays in production

She may not copy it to her laptop to reproduce the bug. That is not negotiable.

MIDDAY

By lunchtime the day belongs to other people.

10:15

A favor

Another team asks her to fix a flaky test in their repository. She says no.

10:40

A page

Checkout is failing. Not her service. She is secondary on the rotation.

11:00

The loop

Push, wait twelve minutes, read a review comment, fix, push again. Twice.

12:30

A branch left behind

Her second attempt is still sitting there, half-finished, and she forgets it.

13:30

The queue

Five pull requests are waiting. There is one reviewer, and he is asleep.

AFTERNOON

Maya loses the afternoon to being asked how the afternoon is going.

15:30

The spiral

482 sits. Someone asks for status. She stops to answer. It sits longer.

15:50

She stops answering

She declines the fourth request and batches every reply until four o'clock.

16:00

The window closes

The reviewer wakes at three. It is not enough. 482 does not land today.

16:30

One thing held

Main was deployable every single minute of the day, including this one.

17:00

A small realization

Nothing merges on Thursdays either. She checks. Nobody planned that.

22:40 · THE REVEAL

You just watched a system run for sixteen hours.

It had a goal it did not meet, and one hour that decided whether it would. Two things fed themselves in circles, one of them in her shower. Something invisible held the whole thing up until nine o'clock, when it stopped. One promise never broke, on the day everything else did.

The reviewer woke at three and the window shut at four. Everything Maya did in the sixteen hours around that hour reached the outcome only through it — and for twenty of those sixty minutes she was answering questions about 482 instead of standing ready to act on whatever came back about it.

You have watched all ten. You do not yet have the words.

01

PART ONE

Now we give each of those things its name.

WORD ONE · SYSTEM

A system is parts, connected, doing something together.

Maya's morning is one. The alarm, the shower, the train, the laptop and the registry all combine to produce a thing none of them produces alone: Maya at her desk, able to work, at nine.

IN MAYA'S DAY

Every part of her morning was useless alone, and the morning still put her at a desk at nine.

LISTING THE PARTS IS EASY

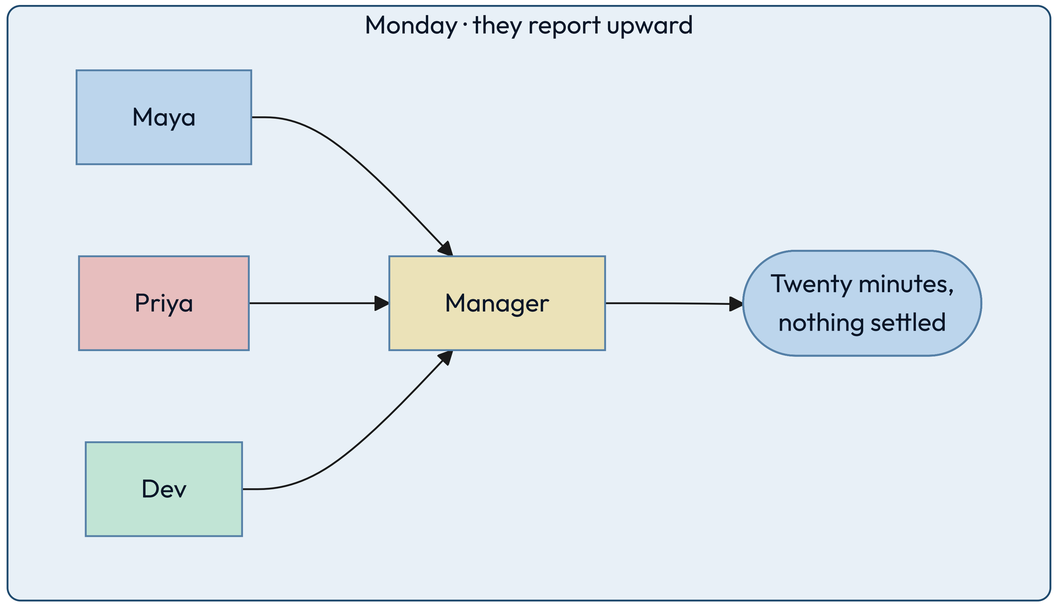
Phone, badge, laptop, transit card. Anyone can write that list.

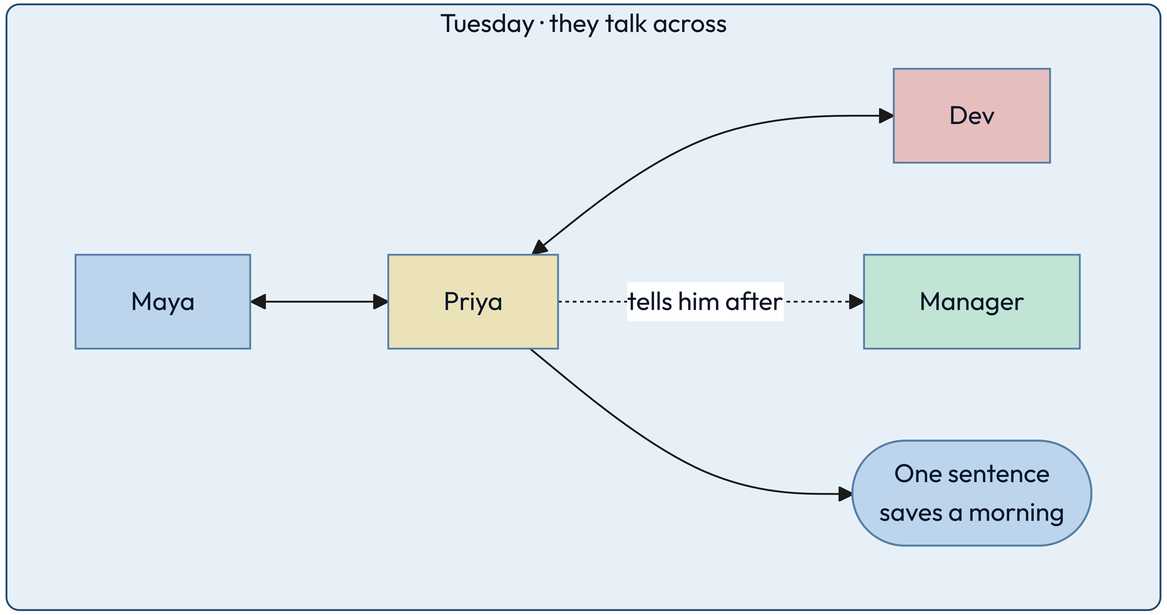
THE CONNECTIONS ARE THE SYSTEM

Change what depends on what, and the behavior changes with the same parts.

09:15 · THE SAME FOUR PEOPLE

Rewire a system without changing a single part, and it behaves differently.





WORD TWO · PURPOSE

A purpose becomes visible at the moment you miss it.

Maya wrote it on a sticky note at half past nine: get 482 merged before four. At four o'clock it had not merged, and the gap between those two facts is the clearest thing in her whole day.

IN MAYA'S DAY

One sentence at 09:30, one outcome at 16:00, and the distance between them.

WRITE IT DOWN OR YOU WILL DRIFT

An unwritten purpose gets quietly replaced by whatever arrived most recently.

A SYSTEM'S PURPOSE IS WHAT IT DOES

Not what its owners say it does. Watch the outputs, not the mission statement.

WORD THREE · BOUNDARY

The boundary is the line between what you change and what you ask for.

At quarter past ten another team asked Maya to fix a flaky test in their repository. She said no. That refusal is the boundary, and she could feel exactly where it was because saying no was uncomfortable.

IN MAYA'S DAY

She could decline the favor. She could not decline the release window.

INSIDE IS WHAT YOU CHANGE

Your code, your schema, your deploys, the alerts that wake you.

OUTSIDE IS WHAT YOU NEGOTIATE

The reviewer's time zone, the registry, the build, another team's repository.

WORD FOUR · ENVIRONMENT

The environment is everything that arrives without asking.

A signal failure held her train for nine minutes. A page pulled her into an outage in a service she does not own. Neither was load, neither was a bug, and neither was hers.

IN MAYA'S DAY

Two arrivals, one benign and one hostile, neither of them load, and no say over either one.

IT IS NOT THE SAME AS LOAD

Regulation, a partner's outage and a colleague's holiday are environment too.

YOU DESIGN FOR IT, NOT AGAINST IT

You cannot stop the page. You can decide what happens to 482 when it comes.

WORD FIVE · PROCESS

A process is a repeatable transformation, and it runs at a rate.

Push, wait twelve minutes for the build, read a comment, fix, push again. Maya ran that loop twice. Every process has inputs, a transformation, outputs, a rate, and something it leaves behind.

IN MAYA'S DAY

Two full loops at twelve minutes each, plus a branch nobody deleted.

THE RATE IS PART OF THE DEFINITION

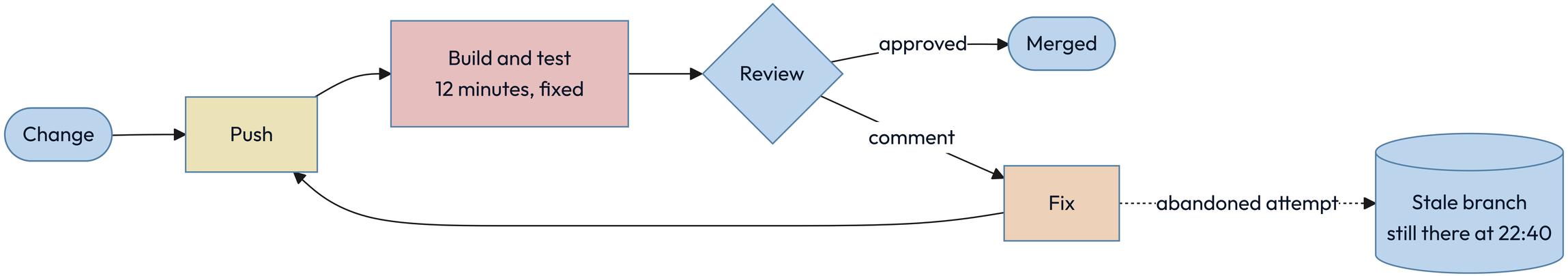
"Run the tests" is not a process until you say twelve minutes, and how often.

LEFTOVER STATE IS WHERE BUGS LIVE

Her abandoned branch is the same shape as a half-applied database migration.

11:00 · THE LOOP, DRAWN

Every process leaves something behind, and that is the part nobody draws.



WORD SIX · MODEL

A model is a deliberate simplification, and it is useful because it is wrong.

The transit map Maya read at five past eight is geographically false. Distances are invented, angles are fiction, the river is the wrong shape. She has never once been lost using it.

IN MAYA'S DAY

A map that lies about distance, tells the truth about order, and never loses her.

IT KEEPS WHAT ANSWERS ONE QUESTION

"Which line, which direction, how many stops." It throws away everything else.

NAME WHAT YOU LEFT OUT

An omission you cannot state is not a simplification. It is a bug you have not found.

WORD SEVEN · CONSTRAINT

A constraint is a limit that removes options rather than adding caveats.

Four of them bounded Maya's day, and each one came from a different place: a build that takes twelve minutes, a team of three, a sleeping reviewer, and a rule about production data.

IN MAYA'S DAY

Physical, economic, human and legal limits, all four of them landing before ten in the morning.

REAL CONSTRAINTS CARRY NUMBERS

"The build is slow" is a complaint. "Twelve minutes" is something you can design against.

SOME YOU CHOSE, AND CAN REVISIT

A team of three is a decision. The speed of the build is arithmetic.

WORD EIGHT · INVARIANT

An invariant is a claim that stays true on the day everything else fails.

Main was deployable every minute of Maya's Tuesday. It was true while the registry was down, while she was paged, and at four o'clock when 482 did not land.

IN MAYA'S DAY

She missed her goal and broke no invariant, and those are two different kinds of bad day.

WRITE THEM AS SENTENCES

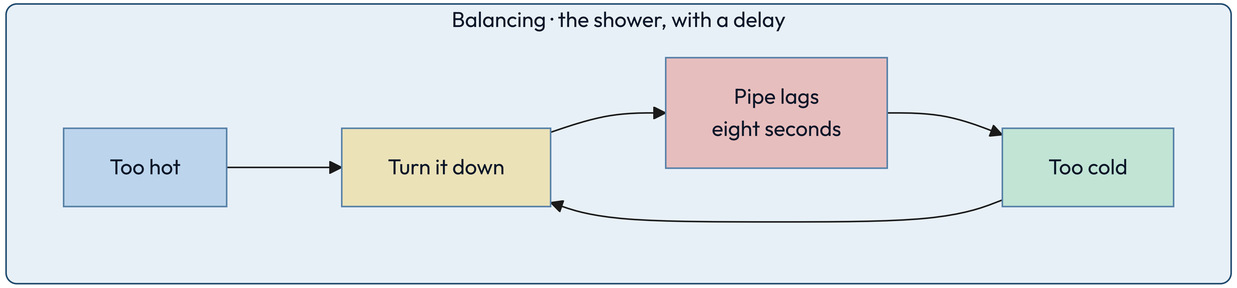
"Main is always deployable." "A balance is never negative." Then name the alert.

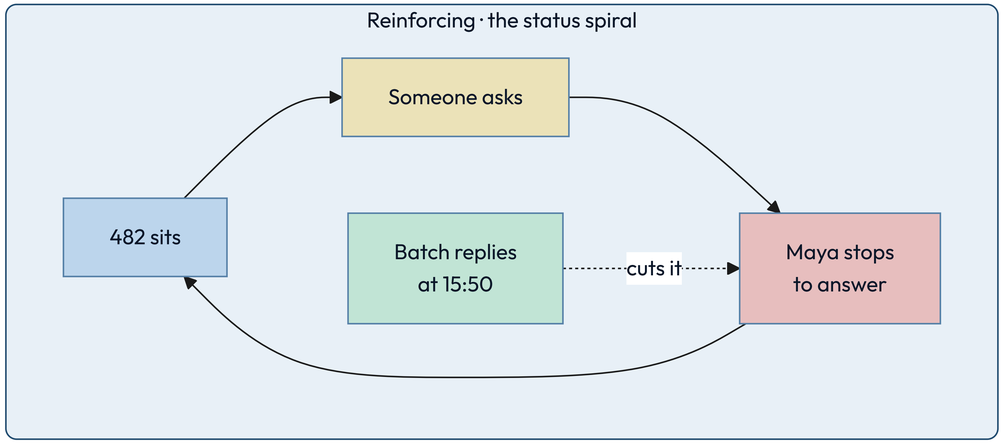
THEY CONSTRAIN OTHER PEOPLE

A real invariant changes what your teammates are allowed to merge.

06:55 AND 15:30 · TWO LOOPS

Two loops look identical until you find the sign.





✦ *A balancing loop pushes back toward a target, and the eight-second delay in the pipe is exactly why she overshoots. A reinforcing loop pushes harder in the direction it is already going.*

Infrastructure is what you only notice on the day it stops.

Wifi, the VPN, the package registry, the build fleet, the identity provider. Maya used every one of them before nine o'clock and thought about none of them until 09:05, when the registry went down and three teams stopped.

THE TEST

If it vanishing stops several unrelated things at once, it is infrastructure.

BORING IS THE REQUIREMENT

It earns its place by being predictable, never by being interesting.

IT IS SOMEONE ELSE'S SYSTEM

Your infrastructure is another team's product, with its own boundary and invariants.

17:00 · THE LAST WORD

Emergence is a pattern the parts fall into, that none of them contains.

Nothing merges on Thursdays. Maya checked six weeks. It is a rhythm, and nobody built a rhythm.

Three things feed each other rather than add up. Work that misses the window waits at the front of tomorrow. A review that comes back with comments sends the same change to the back of the queue. And the queue is served one hour a day. Monday's misses are Tuesday's queue, so the backlog climbs all week until it is bigger than one hour can clear — and the weekend empties it, which is why it happens again next week.

No policy names Thursday. You find it by watching the queue for six weeks.

Five words describe the thing itself.

You did not learn these from a definition. You watched each one happen first, which is the only order that sticks.

01	**System**	Parts, connected, with a purpose.	The whole morning.
02	**Purpose**	What it does, not what it says.	The gap at four o'clock.
03	**Boundary**	What you change, not ask for.	The favor she declined.
04	**Environment**	What arrives uninvited.	The page.
05	**Process**	A transformation with a rate.	Push, build, review.

Three more are the tools you think with.

You never handle the system itself. You handle a drawing of it, its limits and its promises — standing on infrastructure you notice only when it stops.

01	**Model**	A simplification you chose.	The transit map.
02	**Constraint**	A limit that removes options.	Twelve minutes.
03	**Invariant**	What must never go false.	Main is deployable.

THE TRANSLATION

Every move in Maya's day has a name in software.

IN HER TUESDAY	IN A SYSTEM	WHAT IT DECIDES
Five pull requests, one reviewer	The bottleneck	Where any improvement has to land
Declining the fourth status ask	Admission control	Whether load sheds or the system collapses
A twelve-minute build	A fixed cost per attempt	How many attempts a day can hold
The abandoned branch	Leftover state	What a retry finds when it arrives
Nothing merges on Thursdays	Emergence	What no single owner can fix

02

PART TWO

Every design starts with someone who wants something, and something in the way.

THE FRAME

Name the person and the force, and the design starts talking.

Juniors reliably skip both. They write "users" instead of one person with one goal, and they write "scale" instead of a force with a number on it. Neither produces a single design decision.

The protagonist is who the system is for. The antagonist is what makes serving them hard. Everything downstream — the purpose, the boundary, the invariants, the first move — falls out of that pair.

THE DRILL

Two sentences, ninety seconds, and four things you no longer have to guess.

```
Protagonist:  <name> wants to <do X> so that <Y>.
Antagonist:   But <W> — and W is <a number>.

  Purpose       one sentence: what counts as this working
  Boundary      what is mine when W happens, and what I only ask for
  Invariant     what must stay true or <name> stops trusting this
  First move    what W does to the cheapest design that could work
```

✦ *Run it on a turnstile, then a cash machine, then the thing on your screen right now. It gets fast.*

THE DRILL, ON TUESDAY

Maya is the protagonist of her own day, and the interruptions are the antagonist.

MAYA WANTS 482 MERGED BEFORE FOUR O'CLOCK

So that a bug affecting real users stops affecting them, and so she is not carrying it into next week.

BUT SHE IS INTERRUPTED ABOUT HALF A DOZEN TIMES

Not once by anything unreasonable. A page, a favor, four status requests. Each costs the reload, not just the minute.

THE TRAP

Most systems have several protagonists, and you must say which one loses.

Instagram has at least four: the reader opening the app, the ordinary poster, the celebrity with five hundred million followers, and the engineer carrying the pager. They want incompatible things.

Naming one as the protagonist is not a slogan, it is a decision about who waits. Say who you are not designing for, out loud, and half the arguments later in the design stop happening.

THE JOIN

The antagonist chooses which kind of solution you are allowed to build.

THE ANTAGONIST IS…	YOU ARE BUILDING	BECAUSE
Nobody knows if they exist	An MVP	The risk is demand, not load
Growth — the protagonist multiplies	A scaled system	The limit is real and namable
Cost on a path you have profiled	An optimized system	The measurement came first
Physics — you are near a real bound	An optimal system	Only here is a proof worth it
A person who wants in	Security work, at any rung	It is not a rung, it is a floor

INSTAGRAM · THE CASTING

Our protagonist reads on a phone and gives us about a second.

She opens the app a dozen times a day, on a cellular network, usually while doing something else. She wants photographs from the people she chose, recent, ranked, and hers. The poster is a supporting character: he tolerates a spinner, and every time the design has a choice, work goes onto his side.

THE READER'S DEMAND

A fresh, personal page in under a second, twelve times a day, from anywhere.

THE POSTER CAN WAIT

Uploading, transcoding and delivery may all take their time. Nobody is watching.

CASTING DECIDES THE DESIGN

It is why the feed is built when someone posts, not when someone reads.

The antagonist is not scale. It is the shape of the follow graph.

Scale is a quantity, and quantities are boring: you buy more machines. The thing you cannot buy your way out of is a distribution. Follower counts are heavy-tailed, so the average is a lie and the far tail sits six orders of magnitude from the middle.

THE NUMBER THAT MATTERS

The median account has a few hundred followers. The largest has five hundred million.

ONE ALGORITHM CANNOT SERVE BOTH

Nothing else in this design spans six orders of magnitude. That is why there are two paths.

HOLD ON TO THIS

Every hard decision in Part five is this one fact again.

03

PART THREE

"Design Instagram" is five different questions.

BEFORE THE BOXES

Before you design anything, say which kind of answer is wanted.

The same words — design a photo sharing app — have five legitimate answers that share almost no architecture. A build that takes a week and a build that takes two years are both correct, for different questions.

Gall's law says it plainly: a complex system that works is invariably found to have evolved from a simple system that worked. You climb this ladder. You do not parachute onto it.

Five rungs, and each one costs more and commits harder than the last.

You move up when the rung you are standing on stops paying, and you move up one rung at a time.

01	**MVP**	Buys information fast.	Does anyone want this?
02	**Scaled**	Holds up as load grows.	Will it survive success?
03	**Optimized**	Cuts cost on a known path.	Where is the money going?
04	**Optimal**	Provably best, for one model.	What is the real limit?
05	**Specialized**	An advantage nobody can copy.	What can only we do?

RUNG ONE · MVP

An MVP is an experiment wearing a product's clothes.

Its job is to produce a decision, not to last. Every hour spent making it durable is an hour spent on a system you may correctly delete next month.

THE TELL

The riskiest thing is whether anyone wants it, and being wrong is cheap.

BUY SIMPLICITY, NOT CAPACITY

One database, one service, one region, boring technology, nothing clever anywhere.

NAME THE EXIT IN ADVANCE

Write down the number that means "stop, this now has to be built properly."

RUNG TWO · SCALED

A scaled system survives ten times the load without a rewrite.

You have stopped buying information and started buying headroom. The question moves from "does it work" to "what breaks first, and how do I move that limit."

THE TELL

Growth is real, and the current design has a ceiling you can point at.

A BIGGER BOX FIRST, THEN AXES

Vertical scaling is reversible and buys years. Statelessness, caching and queues come next. Partitioning is the one you cannot undo cheaply, so it goes last.

COST PER UNIT STARTS COUNTING

Cost per request stops being noise and becomes the second constraint on every choice.

RUNG THREE · OPTIMIZED

Optimizing means moving one measured number on one hot path.

Optimization without a profile is decoration. You need the measurement first, the target second, and a willingness to accept the complexity you are about to add.

THE TELL

A profile shows which tenth of the work is most of the cost.

PREMATURE IS HALF THE QUOTE

Knuth also wrote that we should not pass up the critical three percent. Both halves.

COMPLEXITY IS THE INVOICE

Each optimization narrows the assumptions the system is allowed to break.

RUNG FOUR · OPTIMAL

Optimal means provably best against an objective you wrote down.

Rarer than it sounds. It needs a stated objective, a stated model, and a proof or a bound — and it is optimal only for the assumptions you fixed in place.

THE TELL

The objective is a function and the constraints are inequalities, on paper.

THE PROOF IS AGAINST A MODEL

Change the assumptions and the optimal answer changes underneath you.

IT AGES BADLY

An optimal design pinned to last year's hardware is a legacy system with a certificate.

A specialized system trades generality for something nobody can copy.

Custom silicon, a purpose-built storage engine, a scheduler that knows your physics. You give up flexibility, portability and hiring pool for a capability the market cannot sell you.

THE SIGNAL

The advantage is durable, measurable, and central to why customers choose you.

THE COST NEVER ENDS

You now maintain what everyone else gets free from a vendor, forever.

ALMOST NOBODY IS HERE

Most teams reaching for this rung needed the optimized one and got excited.

THE RULE

Climb one rung at a time, and only on evidence.

MOVE UP WHEN THE CURRENT RUNG FAILS ON A MEASUREMENT

A number, a profile, a named risk. Not a feeling that things are getting big.

MOVE UP EXACTLY ONE RUNG

Jumping from MVP to optimal buys rigor for assumptions nobody has tested yet.

MOVE BACK DOWN WHEN THE EVIDENCE CHANGES

A rewrite that simplifies is a legitimate move, not an admission of anything.

04

PART FOUR

Six kits, and every entry answers the same three questions.

HOW TO READ A KIT

Reach for it when. Walk away when. The constraint you inherit.

One shape for every entry, so you can hold two options side by side without re-reading a manual. Each kit opens with a diagram of the patterns it covers, and closes with the invariants that hold across all of them.

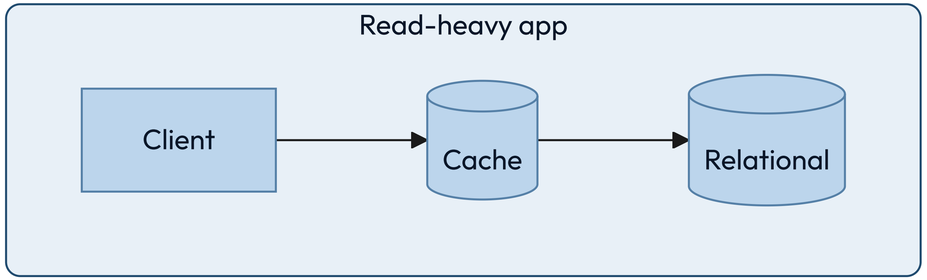
The entries name concepts, not products. Products turn over every few years. The constraint that an append-only store puts on your reads does not.

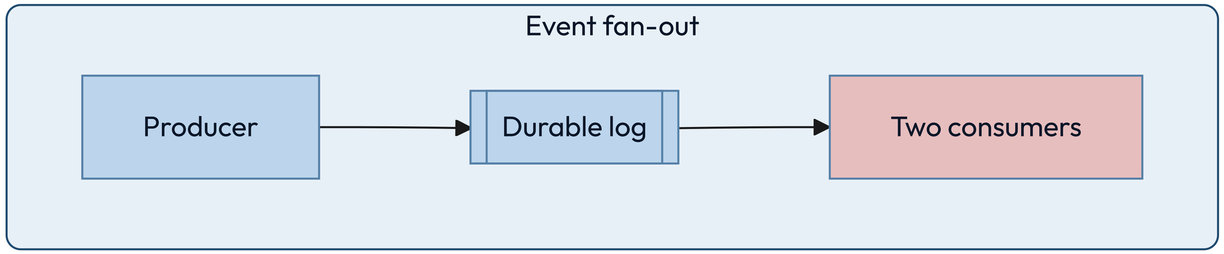
DATA

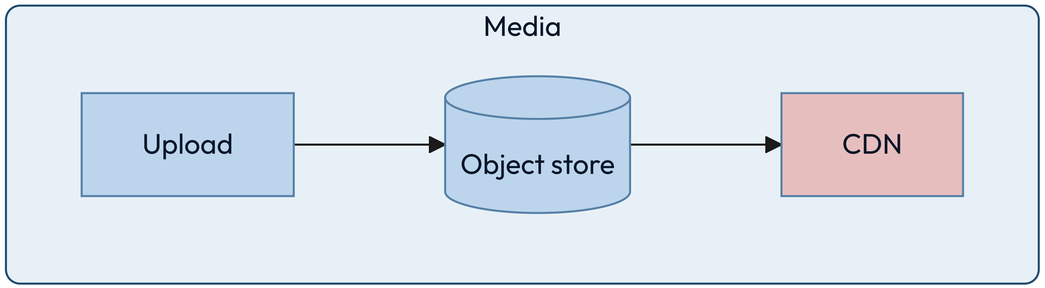
Every storage choice is a bet about how you will read it later.

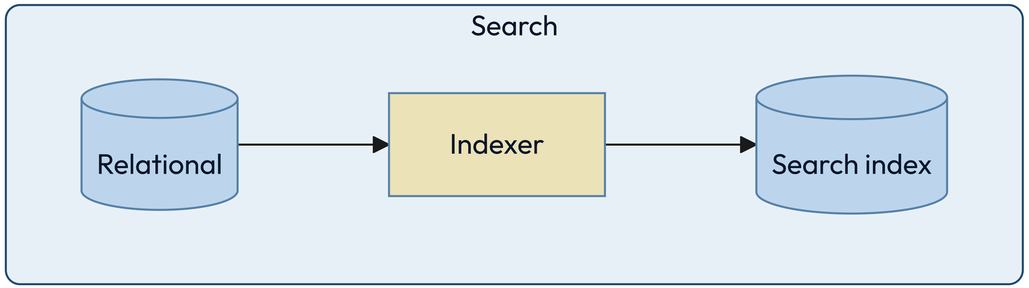
DATA KIT · THE PATTERNS

Four shapes cover almost everything you will store.









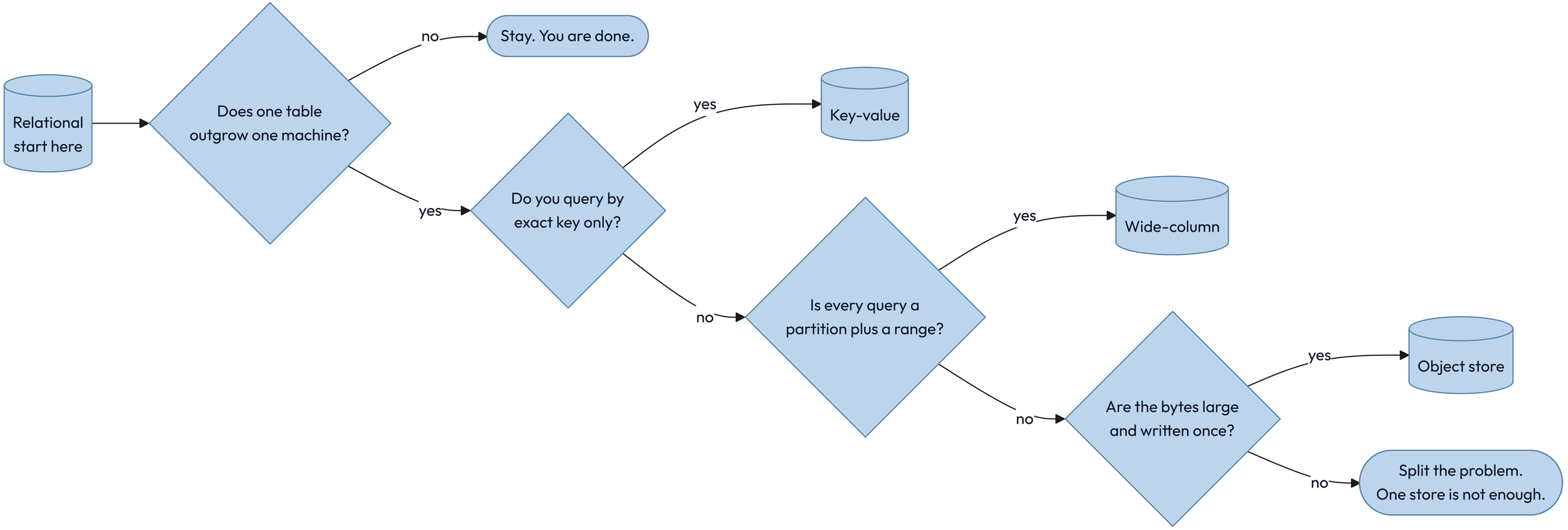
Five questions pick a store, and the product name comes last.

Answer these before anyone says a brand. Most database arguments are really a disagreement about question two.

01	**Shape**	How the data is structured.	Rows, documents, or edges?
02	**Access**	How it is read and written.	Lookups, ranges, or scans?
03	**Consistency**	What a reader may see.	Must it be the latest?
04	**Scale**	Size, throughput, growth.	Does it fit one machine?
05	**Cost**	Money, operations, skill.	Who runs it at 3am?

DATA KIT · THE DECISION

Start at relational and branch out, because that is how anyone decides.



DATA KIT · RELATIONAL

A relational store is the default until you can name why it is not.

Reach for it when

Entities relate, writes touch several at once, and the questions will keep changing.

Walk away when

One table outgrows one machine's writes, or the schema genuinely differs per row.

The constraint you inherit

Joins and transactions need a coordinator. Self-sharding loses both; a distributed engine charges a round trip.

DATA KIT · KEY-VALUE

A key-value store is a hash map with an operations team.

Reach for it when

You always know the exact key, and handing the value back is the store's only job.

Walk away when

You need to ask any question at all about what is inside the value.

The constraint you inherit

There is no second way in. Every new query means a new key you write and maintain yourself.

DATA KIT · DOCUMENT

Choosing a document store means choosing to keep one object whole.

Reach for it when

A read fetches one self-contained object, and the shape varies between records.

Walk away when

The same fact lives in many documents and has to stay consistent across them.

The constraint you inherit

Denormalized copies. Every update to a shared fact becomes a fan-out — one write you now owe to many places.

DATA KIT · THE SCAN

Five columns, applied to the five stores that can hold truth.

STORE	ACCESS	CONSISTENCY	SCALES BY	WEAK AT
Relational	Key, range, join	Strong on the leader	Replicas, then partitioning	Cross-shard writes
Key-value	Exact key	Tunable	Horizontal	Any other question
Document	Key, secondary index	Per document	Horizontal	Cross-document facts
Wide-column	Partition plus range	Tunable	Horizontal	New query patterns
Graph	Traversal	Engine-specific	Poorly	Partitioning at all

DATA KIT · WIDE-COLUMN

A wide-column store buys enormous write throughput and freezes your queries.

Reach for it when

Writes are relentless, and every read is a partition key plus a sorted range.

Walk away when

Query patterns are still moving, or you need a transaction across partitions.

The constraint you inherit

The primary key is the schema. Changing how you query means rewriting the data.

DATA KIT · GRAPH

A graph store is for questions that traverse, not questions that filter.

Reach for it when

The query walks relationships of unknown depth: reachability, paths, recommendations.

Walk away when

You have relationships but only ever join two hops. A relational store does that faster.

The constraint you inherit

Traversals resist partitioning, so scaling out is genuinely harder here than anywhere else.

HALFWAY

Five stores hold truth, and two of the derived ones secretly do too.

An object store and a time-series store are usually where a fact first lands — nothing regenerates a photograph or last Tuesday's CPU samples. They are sources of truth wearing the clothes of a derived tier.

The genuinely derived stores are the search index, the vector index, the cache and the warehouse. Anything derived must be rebuildable, must be allowed to lag, and must never be the only copy.

DATA KIT · OBJECT STORE

An object store is the cheapest home for a write-once file.

Reach for it when

Items are large, written once, fetched by key, and must survive for a decade.

Walk away when

You need to modify part of an object, or to list and filter by what is inside it.

The constraint you inherit

Listing is slow and expensive. The index of what you stored belongs somewhere else.

DATA KIT · CACHE

A cache converts a correctness problem into a timing problem.

Reach for it when

Reads repeat, the source is expensive, and slightly stale is genuinely acceptable.

Walk away when

Staleness is unsafe, or the working set is larger than the cache will ever be.

The constraint you inherit

Invalidation. You now own a second copy whose wrongness is measured in seconds.

DATA KIT · DURABLE LOG

A durable log turns "do it now" into "do it reliably, soon."

Reach for it when

Producers outpace consumers, or several systems need to see the same events.

Walk away when

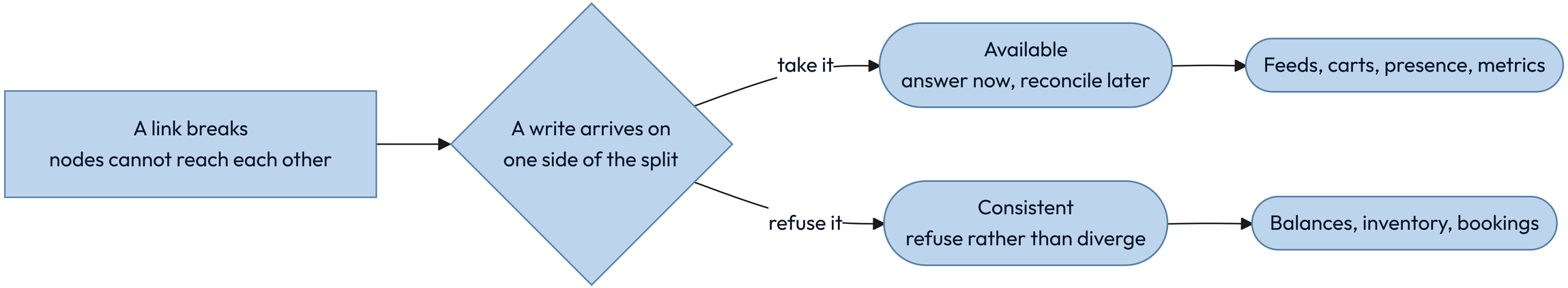
The caller needs the result inside the same request.

The constraint you inherit

At-least-once delivery. Every consumer must be idempotent or you will charge somebody twice.

DATA KIT · UNDER A PARTITION

CAP is a decision you make during a network fault, and only then.



DATA KIT · CONSISTENCY

Three rungs cover almost every argument you will have about consistency.

01	**Linearizable**	Every read sees the latest write. Costs a coordination round trip, every time.
02	**Read your writes**	You always see your own edits. The least a person who just typed something accepts.
03	**Eventual**	Replicas agree in the end. Cheapest, and correct for feeds, counters and presence.

DATA KIT · TWO PHILOSOPHIES

ACID and BASE answer different questions.

Ask whether coordinating now costs less than reconciling later.

I

ACID

Coordinate first, so the data is never observably wrong. You pay in latency and in how far you can partition.

→

II

BASE

Diverge now, converge later. You pay in application code, because every reader must tolerate stale state.

DATA KIT · INDEXES

An index is a second copy of your data, sorted for exactly one question.

Indexes make reads fast by making writes slower and storage larger. Each one is a promise to maintain a sorted structure on every single insert, update and delete, forever.

THE TELL

You can name the exact query each index exists to serve, out loud, right now.

THE BILL ARRIVES ON WRITES

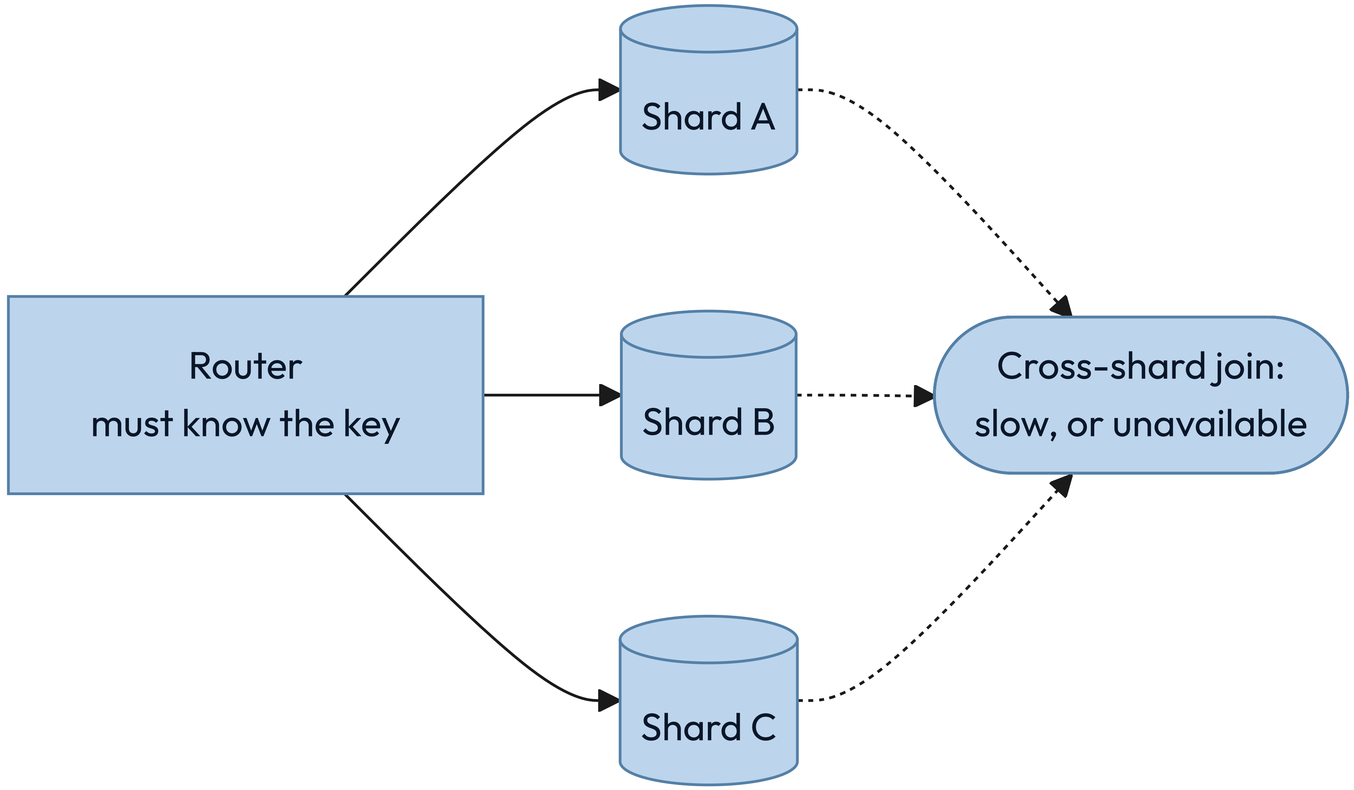
Five indexes mean five extra structures touched per insert, not one.

SELECTIVITY DECIDES, NOT CARDINALITY

Three evenly spread values get ignored. Three where one appears in 0.1 percent do not.

DATA KIT · PARTITIONING

Sharding buys throughput by giving up the questions that cross shards.



DATA KIT · THE SHARD KEY

The shard key is the decision you cannot undo cheaply.

By hash of the key

Spreads load evenly and destroys range queries. The safe default.

By range

Keeps ranges fast and invites a hot spot on the newest range.

By tenant or region

Matches both the access pattern and the law, until one tenant grows enormous.

KEY INSIGHT

Name the query that will now cross shards. If you cannot, you have not chosen a key — you have chosen a hash.

DATA KIT · THE INVARIANTS

Four sentences should hold in any data design. Which does yours break?

01

Exactly one source of truth per fact

Every other copy is derived, and is labeled as derived where people can see it.

02

Every derived store can be rebuilt

If the search index burns down, a job restores it from the source, unattended.

03

Every consumer of a queue is idempotent

At-least-once delivery is the only delivery anyone actually gets.

04

Every write path states its consistency level

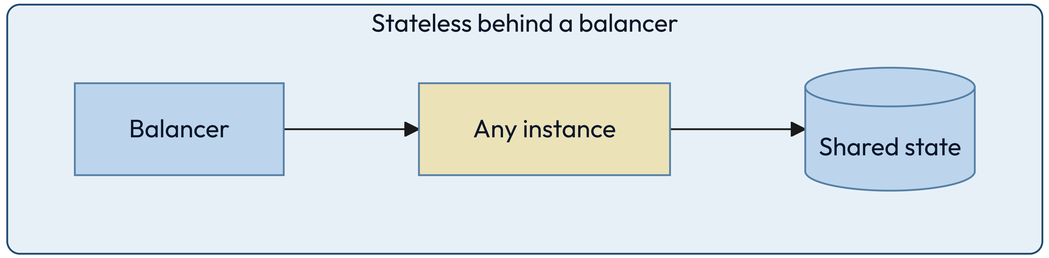
"Whatever the database does by default" is not a stated level.

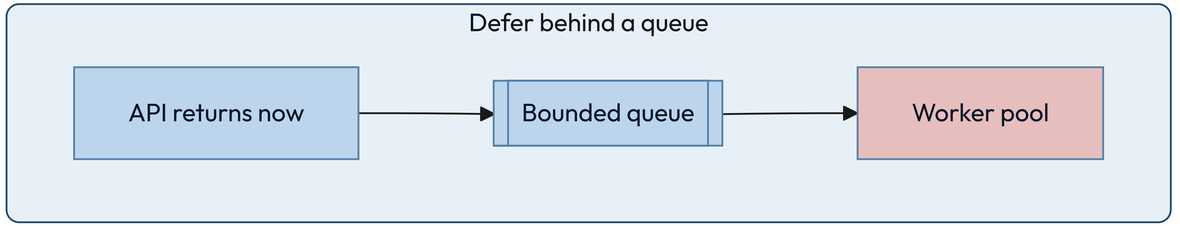
COMPUTE

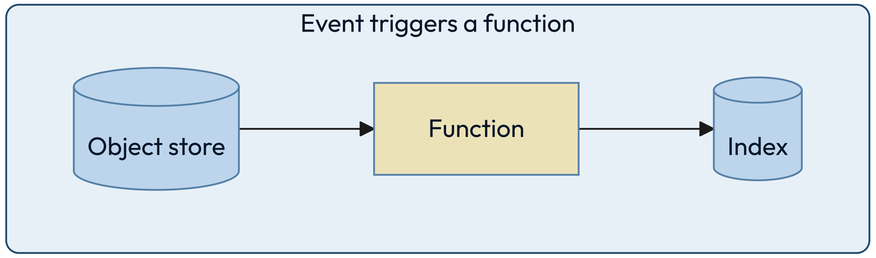
Choosing compute is choosing how much of the machine you still own.

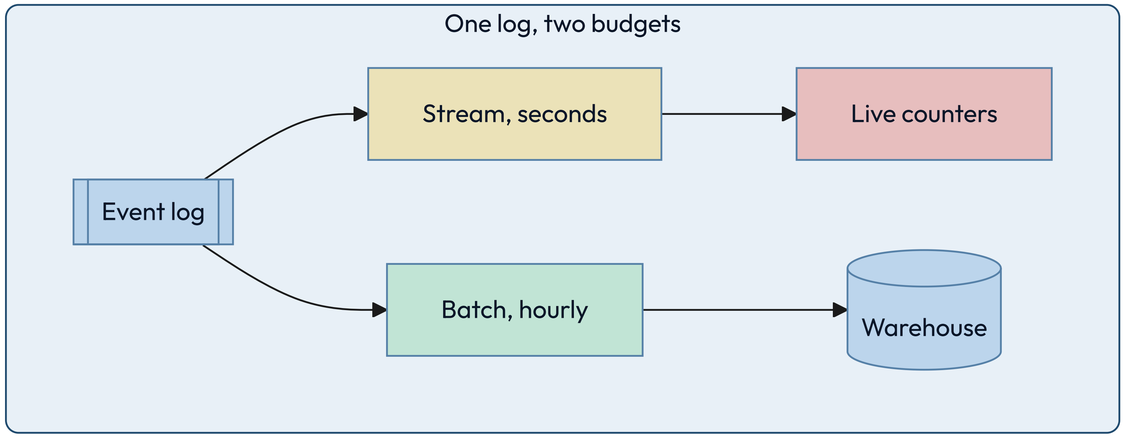
COMPUTE KIT · THE PATTERNS

Each of these hands the machine to somebody else.









COMPUTE KIT · VIRTUAL MACHINES

A virtual machine is what you reach for when the process outlives the request.

Reach for it when

The workload runs continuously, holds state in memory, or needs unusual kernel settings.

Walk away when

Traffic is spiky and idle machines start to dominate the bill.

The constraint you inherit

Patching, capacity planning, and the gap between staging and production are now yours.

COMPUTE KIT · CONTAINERS

A container makes the deployable unit identical everywhere it runs.

Reach for it when

You run many services, deploy often, and want one packaging story across all of them.

Walk away when

You run one small service. An orchestrator costs more than it saves at that size.

The constraint you inherit

The orchestrator is now infrastructure, with its own failure modes and its own pager.

COMPUTE KIT · FUNCTIONS

A function is compute you rent by the millisecond, so idle costs you nothing.

Reach for it when

Traffic is spiky or rare, the work is short, and per-request isolation is welcome.

Walk away when

Runs are long, or steady traffic makes per-request pricing dearer than a machine.

The constraint you inherit

Cold starts, unless you pay to keep instances warm — which is paying for idle again.

COMPUTE KIT · THE PROPERTY

Statelessness is what makes a machine replaceable.

A STATEFUL INSTANCE

It holds something no other instance has: a session, a lock, a warm cache, an open connection. Scaling means moving that state, and a crash means losing it.

A STATELESS INSTANCE

Any instance serves any request because the state lives somewhere else. Scaling is arithmetic and failure is a routing change.

COMPUTE KIT · THE INVARIANTS

Anything you run should satisfy these four. Which does yours break?

01

Any instance can be killed without a customer noticing

If that is false, you have state you have not named yet.

02

Deployments are reversible

A rollback is a routine operation, not an incident response.

03

Capacity is a number somebody owns

Not "it autoscales" — the ceiling, the cost, and who gets paged at it.

04

Startup does not depend on startup order

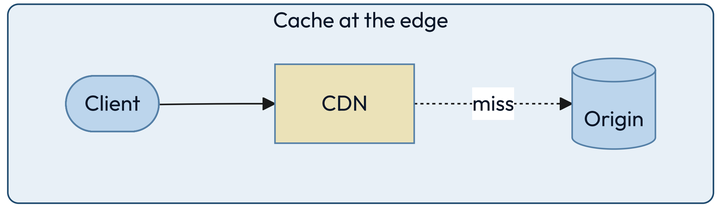
Services retry into each other instead of requiring a sequence.

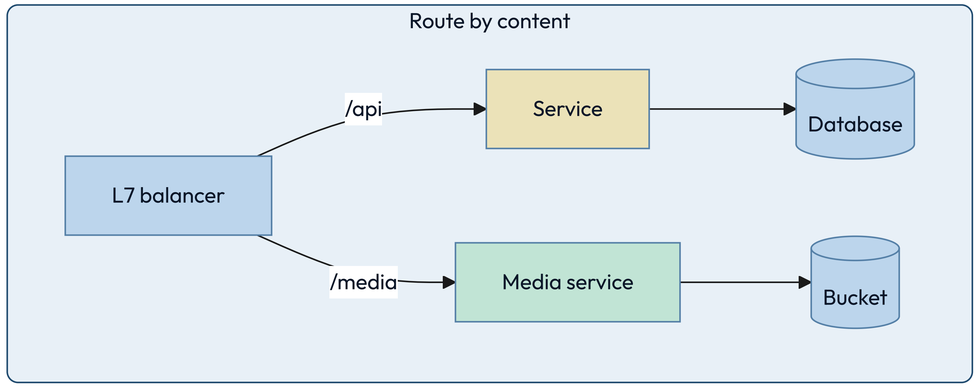
NETWORK

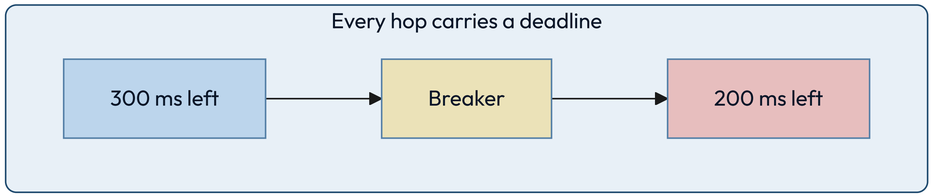
Distance is the one cost you cannot optimize away.

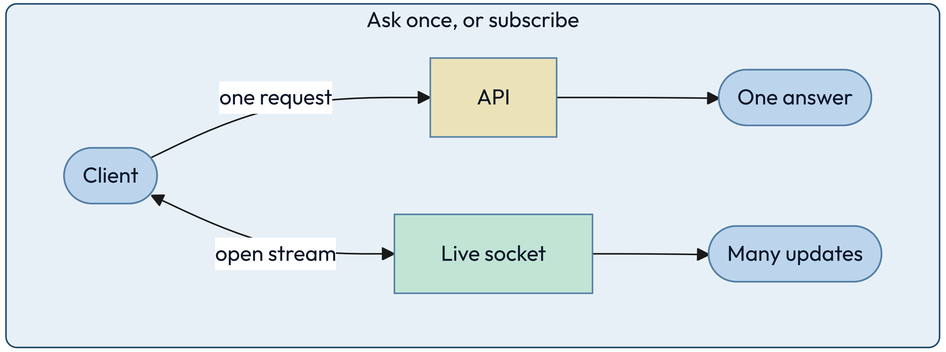
NETWORK KIT · THE PATTERNS

Distance costs you differently in each of these.



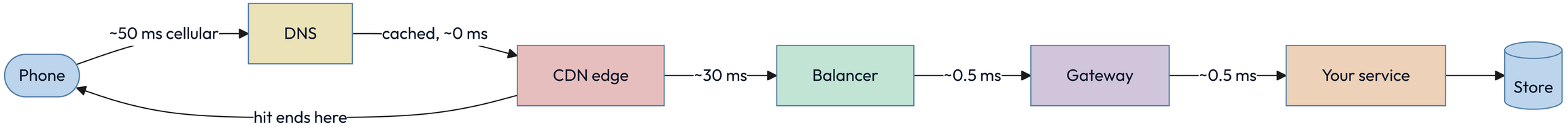






NETWORK KIT · THE REQUEST PATH

Four hops separate a tap from your service, each spending budget.



NETWORK KIT · THE NUMBERS

Three numbers settle most latency arguments before they start.

01	**Memory read**	100 ns
02	**Datacenter hop**	0.5 ms
03	**Cross-continent round trip**	150 ms

WHY THAT NUMBER IS FINAL

Light in fiber covers about 200 kilometers per millisecond.

Nothing changes that. The measured 150 milliseconds is well above the straight-line floor, because packets do not travel in straight lines and every hop queues.

A design that needs three sequential intercontinental round trips has spent half a second before it executes an instruction. Replication, caching and edge delivery all exist to buy that distance back, and none of them makes it free.

NETWORK KIT · THE CDN

A CDN moves bytes closer to readers and moves nothing else.

Reach for it when

The content is large, popular, and identical for many people.

Walk away when

Every response is personal and cacheable for exactly one reader.

The constraint you inherit

Purging is eventual. Put a version in the URL and never invalidate anything again.

NETWORK KIT · TIMEOUTS

A request with no timeout is a resource leak waiting for a bad afternoon.

Every waiting request holds a connection, a thread and some memory. Under a slow dependency, unbounded waits turn one struggling service into a queue of stuck callers — which is the shape of the page that pulled Maya in at twenty to eleven.

THE TELL

Every outbound call has a deadline, and the deadline shrinks as it propagates.

RETRIES NEED A BUDGET

Retrying without a cap turns a brief failure into a flood you built yourself.

BACKOFF MUST BE RANDOM

Synchronized retries arrive together and rebuild the spike you just survived.

NETWORK KIT · THE INVARIANTS

A call that leaves your process owes you these four.

01

Every remote call has a deadline

Inherited from the caller, and always shorter than the caller's own.

02

Every retry has a budget and jitter

Bounded attempts, randomized delays, and a breaker when the target is down.

03

Every write is idempotent or keyed

The network will deliver your request twice. Decide now what that means.

04

Distance appears in the design

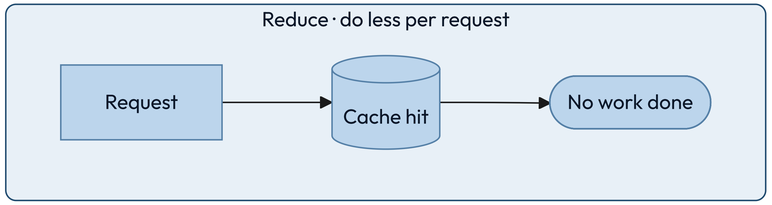
Round trips between regions are counted on purpose, not discovered in production.

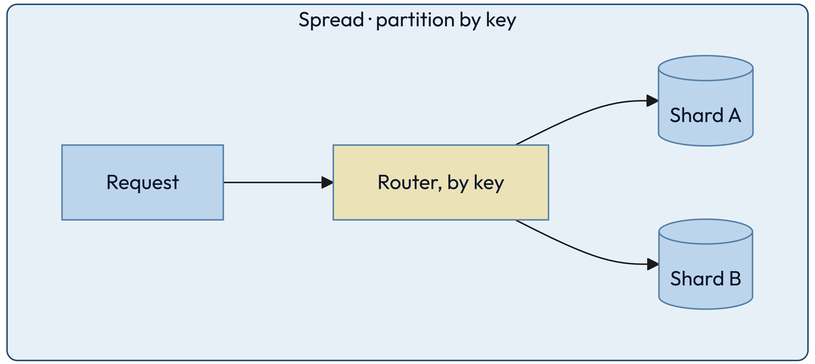
SCALE

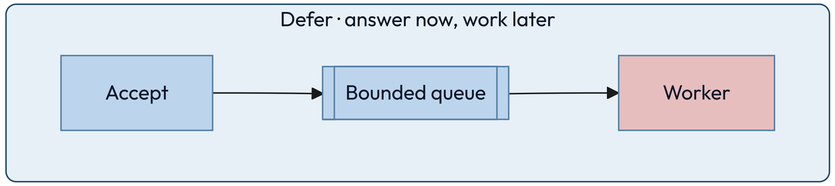
Every scaling change is one of four moves.

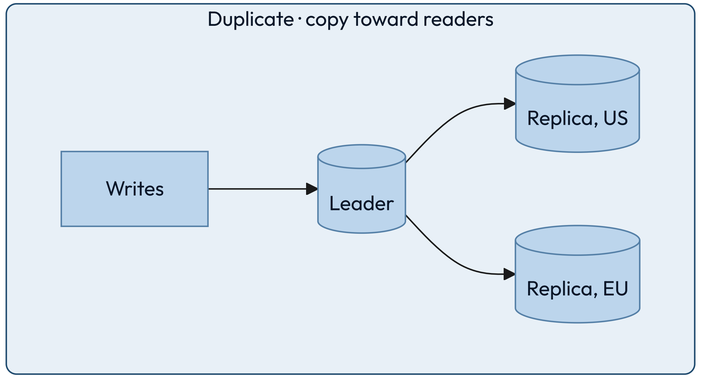
SCALE KIT · THE PATTERNS

The four moves, drawn.









SCALE KIT · THE ONE FORMULA

Little's law ties concurrency, throughput and latency together.

$$L = \lambda W$$

WHERE

- L — requests in flight at once
- λ — arrivals per second
- W — time each one spends inside

USING IT

The formula sizes your thread pool before anyone guesses.

At 2,000 requests per second averaging 50 milliseconds each, 100 requests are in flight at any moment. That is your minimum concurrency, and no tuning makes it smaller while the other two numbers hold.

It also explains the outage. If latency triples during an incident, concurrency triples with it, and a pool sized for the good day is now the thing that fails. Find your own numbers: arrivals on the load balancer, duration in the handler.

SCALE KIT · TAIL LATENCY

The average request is a fiction, and your users live in the tail.

A page that makes 100 parallel calls waits for the slowest one. With a one-percent chance of a slow call, 63 percent of pages hit at least one — that is one minus 0.99 to the hundredth, and you can redo it on a napkin.

THE TELL

You report the 99th percentile per dependency, and the mean nowhere.

FAN-OUT AMPLIFIES IT

More parallel calls turn a rare slow response into a common slow page.

HEDGING BUYS IT BACK

Send a duplicate after the 95th percentile and take whichever answers first.

SCALE KIT · CACHING

Four caching patterns, and the failure each one owns.

Cache-aside

The app fills the cache on a miss. Simple, and it stampedes on a cold key.

Read-through

The cache fetches for you. Cleaner code, and now the cache is on the critical path.

Write-through

Write both together. Always fresh, and every write pays the cache's latency.

Write-behind

Write the cache, flush later. Fastest writes, and a crash loses them.

KEY INSIGHT

Choose by which failure you can survive, not by which pattern reads best in code.

SCALE KIT · IDEMPOTENCY

Idempotency is what makes a retry safe, and retries are not optional.

Networks duplicate, clients retry, queues redeliver. The only question is whether the second delivery is harmless or charges somebody twice.

THE TELL

Every mutating endpoint takes a client-supplied key and deduplicates on it.

THE KEY COMES FROM THE CALLER

Generated before the first attempt, reused on every retry of that same intent.

STORE THE RESULT, NOT THE FACT

A repeat returns the original answer, not an error saying it already happened.

SCALE KIT · THE INVARIANTS

Check these four before you call anything scalable.

01

The bottleneck is named and measured

Not suspected. A number, a graph, and the resource it belongs to.

02

Every queue is bounded

An unbounded queue turns a throughput problem into a memory outage.

03

Load sheds before it collapses

The system rejects work at the edge rather than failing everywhere at once.

04

Adding a machine is routine

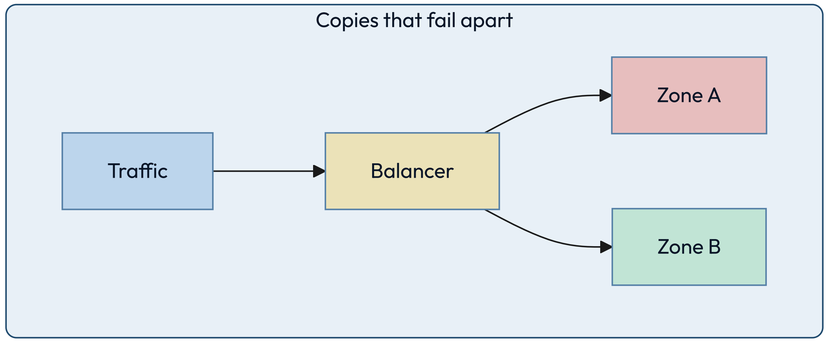
No manual steps, no rebalancing outage, no cold-cache stampede.

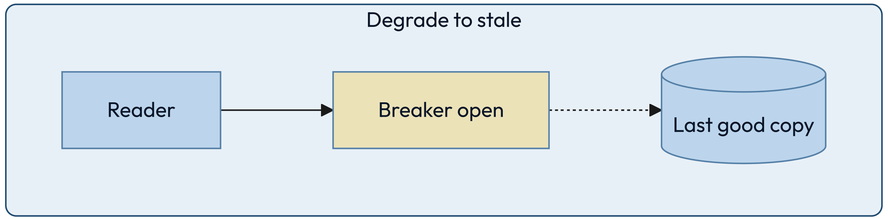
RELIABILITY

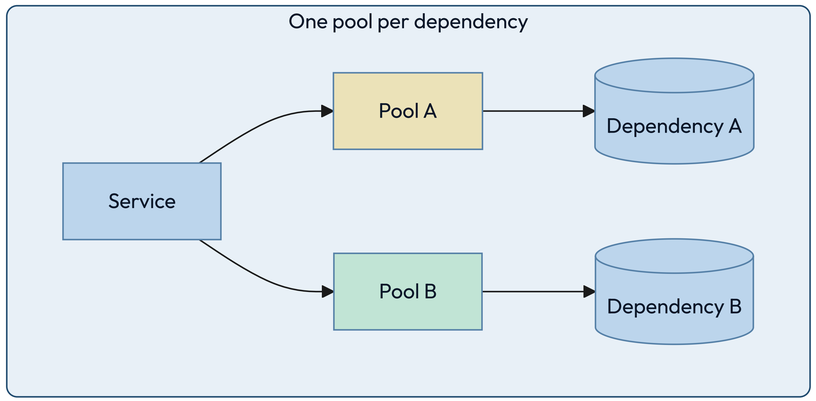
Failure is the environment, not the exception.

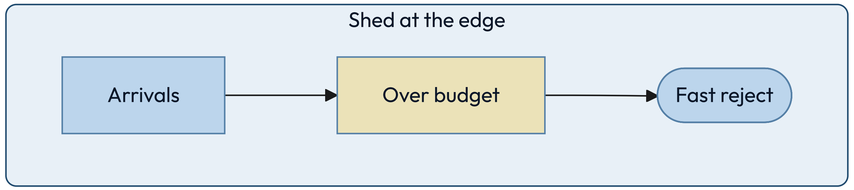
RELIABILITY KIT · THE PATTERNS

Each of these keeps one failure from becoming every failure.



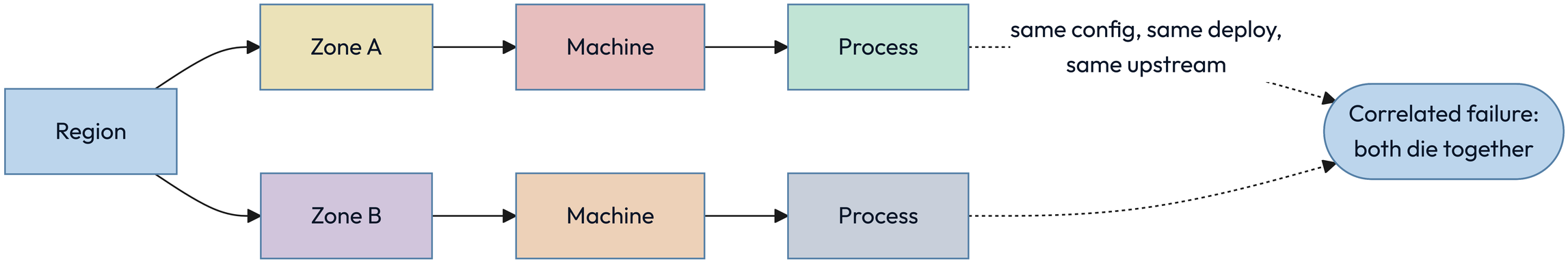






RELIABILITY KIT · FAILURE DOMAINS

Redundancy only helps when the copies can fail apart.



RELIABILITY KIT · CONTAINMENT

Four mechanisms keep a slow dependency from taking the building.

STEP 01

Timeout

Bound the wait, so a slow callee cannot hold your resources.

STEP 02

Retry with backoff

Recover from a blip, with a budget and jitter so you do not amplify it.

STEP 03

Circuit breaker

Stop calling something that is clearly down, and let it recover.

STEP 04

Bulkhead

Give each dependency its own pool, so one queue cannot drain them all.

RELIABILITY KIT · THE OBJECTIVES

Four terms decide what you are allowed to ship this week.

01	**SLI**	The measurement: the share of requests served under 300 milliseconds.
02	**SLO**	Your internal target for it, such as 99.9 percent over 28 days.
03	**SLA**	The external promise, with money attached, and always looser than the SLO.
04	**Budget**	What the objective lets you spend. Exhausted, feature work stops.

RELIABILITY KIT · THE NINES

An availability target is a budget, and it shrinks fast.

01	**99 percent**	3.65 days per year
02	**99.9 percent**	8.8 hours per year
03	**99.99 percent**	53 minutes per year
04	**99.999 percent**	5.3 minutes per year

READING THAT TABLE

Past three nines the humans stop being fast enough.

Fifty-three minutes a year is about one incident with a page, a login, a look at a dashboard and a decision. So four nines does not mean a faster on-call engineer. It means automatic failover and automatic rollback, because a person is no longer in the loop.

Each nine roughly multiplies the cost. Pick the number the business actually needs, and write down what you are choosing not to buy.

RELIABILITY KIT · OBSERVABILITY

Three signals answer three different questions, and none replaces the others.

Metrics

Cheap, aggregate, always on. They tell you that something is wrong.

Logs

Detailed, expensive, one event at a time. They tell you what happened in one case.

Traces

One request across every service. They tell you where the time actually went.

KEY INSIGHT

If you cannot answer "which dependency is slow" inside a minute, you have metrics, not observability.

RELIABILITY KIT · DEGRADATION

A well-designed system gets worse in an order somebody chose.

Under pressure something has to give. Either you decided in advance which features degrade, or the system decides at random and drops the one that takes money.

THE TELL

Every feature has a stated tier, and the lowest tier fails first by design.

READ PATHS OUTLIVE WRITE PATHS

Serving something slightly stale beats serving an error page, for almost every product.

THE FALLBACK IS EXERCISED

An untested degraded mode is untested code running in your worst hour.

RELIABILITY KIT · THE INVARIANTS

Trust nothing in production until these four hold.

01

Every dependency has a defined failure behavior

Written down: degrade, queue, or fail fast. Never "we will see."

02

Redundant copies fail independently

Different zones, different deploys, different upstreams. Otherwise it is one copy.

03

Recovery is practiced

Restores, failovers and rollbacks happen on a schedule, not for the first time at 3am.

04

The system tells you before a user does

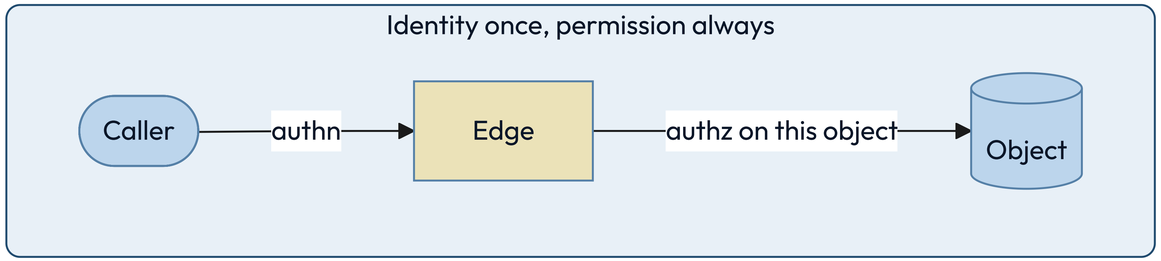
Alerts fire on the indicator, never on the complaint.

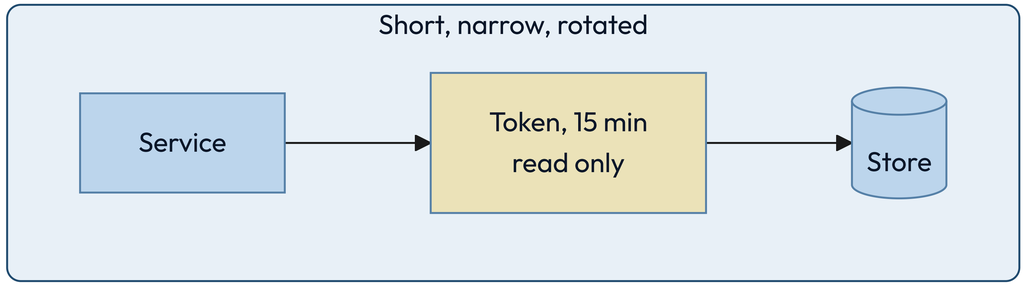
SECURITY

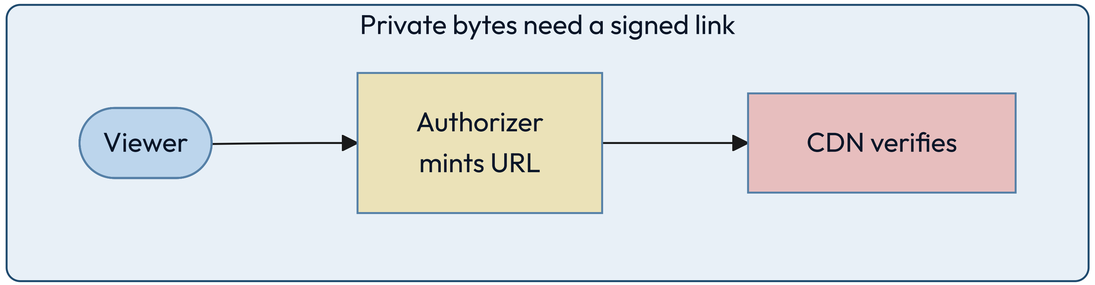
Assume the boundary is already crossed.

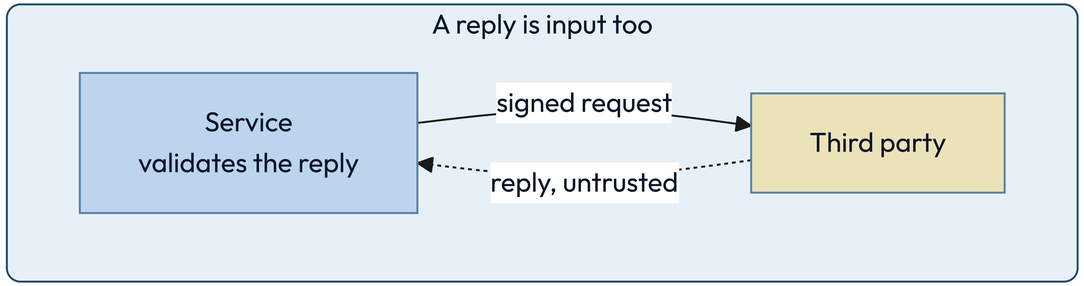
SECURITY KIT · THE PATTERNS

All four of these assume somebody is already inside.



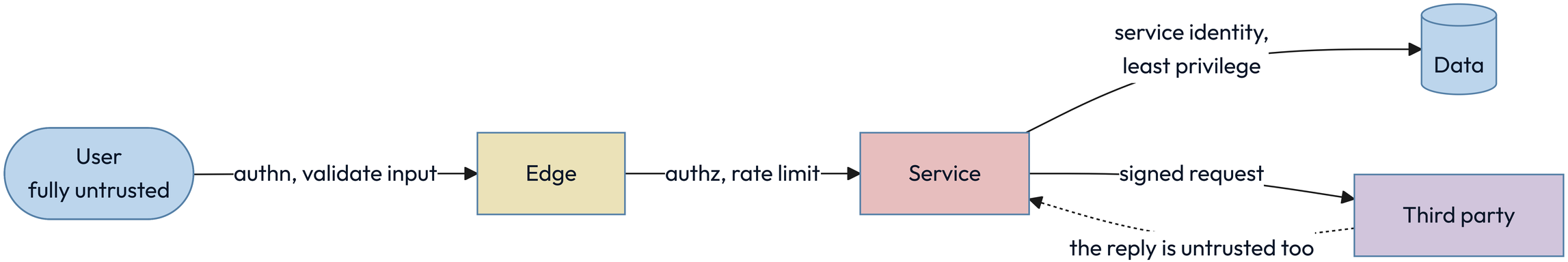






SECURITY KIT · TRUST BOUNDARIES

Every arrow that crosses a trust boundary needs a check on the far side.



SECURITY KIT · THE PAIR

Authentication asks who you are. Authorization asks what you may touch.

AUTHENTICATION

Establishes identity once, at the edge, and hands down a claim anyone downstream can verify. Getting it wrong lets the wrong person in.

AUTHORIZATION

Decides, on every single request, whether this identity may touch this specific object. Getting it wrong lets the right person read somebody else's data.

SECURITY KIT · LEAST PRIVILEGE

Least privilege is measured by what one stolen credential can reach.

Assume any single credential eventually leaks. The design question is not whether that happens; it is how far the person holding it can travel, and for how long.

THE TELL

You can say in one sentence what each service account could reach if stolen.

SCOPE IT AND EXPIRE IT

Narrow permissions, short lifetimes and automatic rotation beat a careful human.

SEPARATE READ FROM WRITE

Most services need one or the other. Almost none needs to delete.

SECURITY KIT · ENCRYPTION

Encryption has three states, and a fourth thing that outranks all of them.

01	**Transit**	TLS everywhere, including between your own services. Networks are untrusted.
02	**Rest**	Disk and backup encryption. Stops a stolen device, not a stolen credential.
03	**Use**	Decrypted in memory to be processed. The hardest state, and the one attacked.
04	**Keys**	A managed store or an HSM, rotated on a schedule, never in the repository.

SECURITY KIT · THREAT MODELING

Six questions find most of the holes before somebody else does.

- ◯ Can someone pretend to be another identity? — spoofing
- ◯ Can data be changed in transit or at rest? — tampering
- ◯ Can an actor deny having done something? — repudiation
- ◯ Can private data reach the wrong reader? — disclosure
- ◯ Can one caller exhaust a shared resource? — denial
- ◯ Can a caller gain rights nobody granted? — elevation

SECURITY KIT · THE INVARIANTS

Sign your name to a design only when these four are true.

01

Every request is authorized against the specific object

Not the endpoint. Object-level checks are where real breaches actually happen.

02

Secrets never enter the repository or a log line

They live in a managed store, are injected at runtime, and they rotate.

03

All input is untrusted, including from your own services

A compromised internal caller is the ordinary case, not the exotic one.

04

Every privileged action is attributable

An immutable record of who, what, when, and from where.

05

PART FIVE

Now we design the thing Maya opened while the train sat.

INSTAGRAM · THE WORKSHEET

Nine fields, filled in before a single box goes on the board.

Protagonist	A reader on a phone. Twelve opens a day, on cellular.
Antagonist	The shape of the follow graph — a tail at 5×10^{8} edges.
Purpose	A fresh, personal, ranked page in under a second.
Boundary	In: feed, posts, edges, media. Out: the phone, the network, the law.
Environment	Spiky traffic, partitions, duplicate deliveries, people who want in.
Constraints	200 ms p99 · 60 reads per write · media durable forever
Invariants	A post reaches eligible followers · media never lost · a like counts once
Bottleneck	Hydration lookups: 70K feed reads/s, but ~5.6M backend reads/s
Solution type	Scaled. Not an MVP, not optimal.

INSTAGRAM · WHAT IT MUST DO

Four operations carry the entire product.

01

Post a photo

Upload media, attach a caption, publish it to the people who follow you.

02

Follow an account

Build the directed graph that decides whose posts you are eligible to see.

03

Read a feed

A ranked, paginated page of recent posts from the accounts you follow.

04

React

Like and comment, with counts visible on every post.

INSTAGRAM · WHAT IT MUST GUARANTEE

Four guarantees decide the architecture more than the features do.

01

The feed serves in under 200 milliseconds

Server-side, 99th percentile, measured from the reader's own region.

02

Reads dominate writes by about sixty to one

At the API. Push fan-out inverts it underneath: 180K feed writes a second against 70K reads.

03

Posts, media and the graph are durable forever

Lose any of the three and nothing rebuilds them. Only derived data rebuilds.

04

Eventual consistency is fine on the feed

A late post and a lagging count are fine. A blank count is not.

INSTAGRAM · THE ESTIMATE, WORKED

Two assumptions and some division give you every number that matters.

Assume 500 million daily users opening the feed twelve times a day, and 100 million photos posted a day. The rest is division: reads are `500M × 12 ÷ 86,400`, or about `70K/s`; writes are `100M ÷ 86,400`, or about `1.2K/s`; bytes are `100M × 2MB`, or `200 TB/day`. That puts the ratio near `60:1`.

The twelve is the only number doing real work. Change the 500 million while holding opens and posts per user fixed, and the ratio does not move. Change how often people post, and it does.

INSTAGRAM · THE ENVELOPE

Five numbers, and the architecture is already arguing with you.

01	**Daily active users**	500 M
02	**Photos per day**	100 M
03	**Average writes**	1.2 K per second
04	**Average feed reads**	70 K per second
05	**Media stored daily**	200 TB

YOUR TURN

Redo it for a smaller product before you turn the page.

Fifty million daily users, opening the app four times a day, posting two million photos. Work out reads per second, writes per second, and the ratio between them.

Do it now, on paper, in under a minute. The answer is on the next slide, and the point is not the answer — it is that you can produce one at all when somebody asks in a room.

THE ANSWER

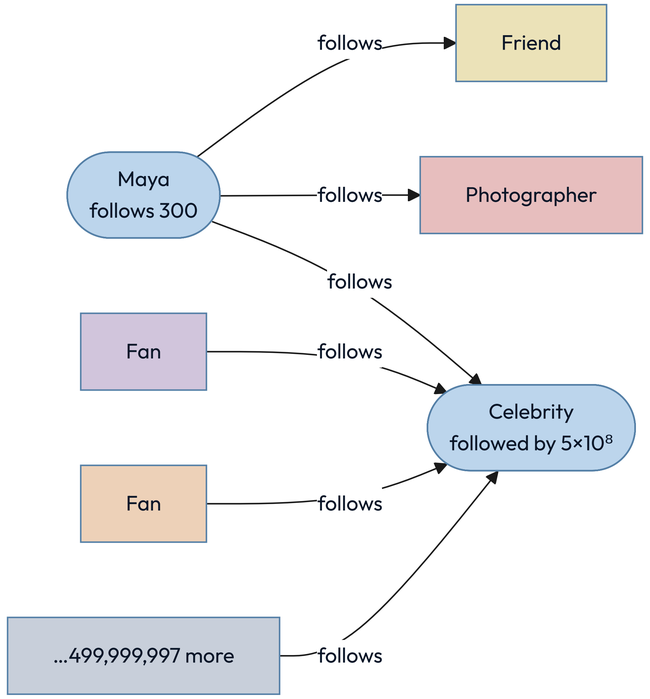
The ratio moved, and the reason it moved is the lesson.

`50M × 4 = 200M reads/day ÷ 86,400 ≈ 2.3K reads/s` · `2M ÷ 86,400 ≈ 23 writes/s` · `ratio ≈ 100:1`

A thirtieth of the read traffic, and the ratio went from 60:1 to 100:1. It moved because you changed how often each person posts, not just how many people there are. The ratio is opens per user divided by posts per user: it survives being wrong about population, never about behavior.

INSTAGRAM · THE GRAPH

The follow graph is directed, and the two directions are not the same size.



✦ *Out-degree is capped at 7,500 by product policy. In-degree is capped by nothing. Every hard problem in this design lives on the right-hand side of this picture.*

Your following list is capped. Your followers list is capped by nothing.

Instagram limits how many accounts you may follow to seven and a half thousand. Nobody limits how many people may follow you. Bounded out-degree, unbounded in-degree — and every hard problem in this design lives on the unbounded side.

FOLLOWING, PER PERSON

At most 7,500. A read of that list is one partition and a sorted range.

FOLLOWERS, PER PERSON

Median a few hundred. Maximum five hundred million. Six orders of magnitude apart.

EVERYTHING AFTER THIS

Is a consequence of that asymmetry. Nothing else in the design spans that range.

INSTAGRAM · THE STORAGE

The edge is stored twice, because it answers two different questions.

```
following                                  followers
  PK  (source_id)                            PK  (target_id, bucket)
  CK  (target_id)                            CK  (created_at DESC, source_id)
  "does A follow B?" is a point read         the follower list, newest first
  capped at 7,500 rows                       bucket = hash(source) % B(target)
                                             B = clamp(followers / 1e6, 1, 512)
```

✦ *The forward edge is the source of truth; the reverse is materialized from the log and repaired by a background job.* `B` *may only ratchet upward — shrink it and edges become unreachable — and because early edges were written under a smaller* `B`*, the low buckets carry several times the average.*

INSTAGRAM · WHAT EACH READ COSTS

Four of these reads are cheap. The fifth is the whole problem.

THE READ	THE PATH	WHAT IT COSTS
Does A follow B?	`following (A)` then `B`	One point read
Who does A follow?	`following (A)` range	7,500 rows, one partition
Who follows B, ordinary B?	`followers (B, 0)` range	~10^4 rows, one partition
Follower count	`user_edge_stats`	One point read, cached
Who follows B, celebrity B?	`followers (B, 0..499)`	**5×10^8 rows, 500 partitions**

LET US GET THIS WRONG

Design it the obvious way: when you post, write into every follower's feed.

The reader then does almost nothing. One lookup, one page, no merging, no ranking across sources. It is fast, it is simple, and for an account with two hundred followers it is unambiguously correct: two hundred small writes, and every reader gets a page in a millisecond.

Hold that design in your head. Now a single account with five hundred million followers posts one photograph. Before you turn the page, predict what happens.

ONE POST, ONE CELEBRITY

25 GB

Five hundred million feed inserts at roughly fifty bytes each, from a single API call.

500 seconds of queue

At a million inserts a second, that is eight minutes serving nobody else.

180K inserts a second

The whole platform's ordinary load: 1.2K posts times about 150 pushed followers.

45 minutes of backlog

One celebrity post is that much of everyone else's work, arriving at once.

ONE SUPER NODE, FOUR FIRES

Writing 25 GB from one call is only the first thing that breaks.

The fan-out queue

25 GB of inserts from one call, in front of everybody else's posts.

The hot partition

Unbucketed, those edges are one partition key on one replica set.

The cache

25 GB of churn evicts the working set of readers who did nothing wrong.

The thundering herd

One cache key, invalidated on publish, missed by a million readers at once.

KEY INSIGHT

A skewed key produces a system where one machine is on fire and the rest are idle.

WHY THE HYBRID IS FORCED

Two cost curves, and the graph decides where they cross.

Pushing costs one write per follower, and the follower count is unbounded. Pulling costs one read per followee, and the followee count is capped at seven and a half thousand. So push is cheap exactly where the unbounded side is small, and pull is cheap exactly where the bounded side is what you walk.

There is a second reason, and it is the better one: a celebrity's recent-posts list is written once and read five hundred million times. It is the most cacheable object in the system. It is the most cacheable object in the entire system.

DECISION

Push below the threshold, pull above it, and merge at read.

The two strategies fail at opposite ends of the same graph, so the design uses each one where the other breaks.

ONE STRATEGY FOR EVERYONE

- Simple to explain, guaranteed to fail at one end. Push drowns on celebrities; pull costs hundreds of reads per page for everyone else.

SPLIT AT THE THRESHOLD

- Push from ordinary accounts, pull from high-follower accounts, merge the two at read time. Ordinary posts land in a page that is already built; the celebrities she follows are fetched on demand and merged in, so no writer ever fans out to millions. The read costs one lookup plus ten to thirty fetches, and each strategy runs where its cost curve is the lower one.

RECOMMENDATION

The threshold is not a preference. It is where two cost curves cross, and the graph draws it.

INSTAGRAM · THE THRESHOLD

Around fifty thousand followers, and it is a rate, not a count.

Fifty thousand puts a fraction of a percent of accounts on the pull path. Readers concentrate on popular accounts, so someone following three hundred typically has ten to thirty above the line.

Run the tail arithmetic on that: at thirty pulls and a one-percent slow call, `1 - 0.99^30` is twenty-six percent of pages hitting at least one. So **cap the pulls** — newest N sources, page the rest — and let the cap bound the tail instead of the graph.

The better predicate is fan-out work per day, not followers: fifty thousand followers posting forty times a day costs more than two hundred thousand posting weekly.

The celebrity is a protagonist of the product and the antagonist of your design.

The product exists partly for her. She is the reason a hundred million people opened the app this morning, and the reason the write path cannot be one code path. Both things are true at once, and holding both is what designing actually feels like.

FOR THE PRODUCT

She is the most valuable account on the platform, and she must post instantly.

FOR THE DESIGN

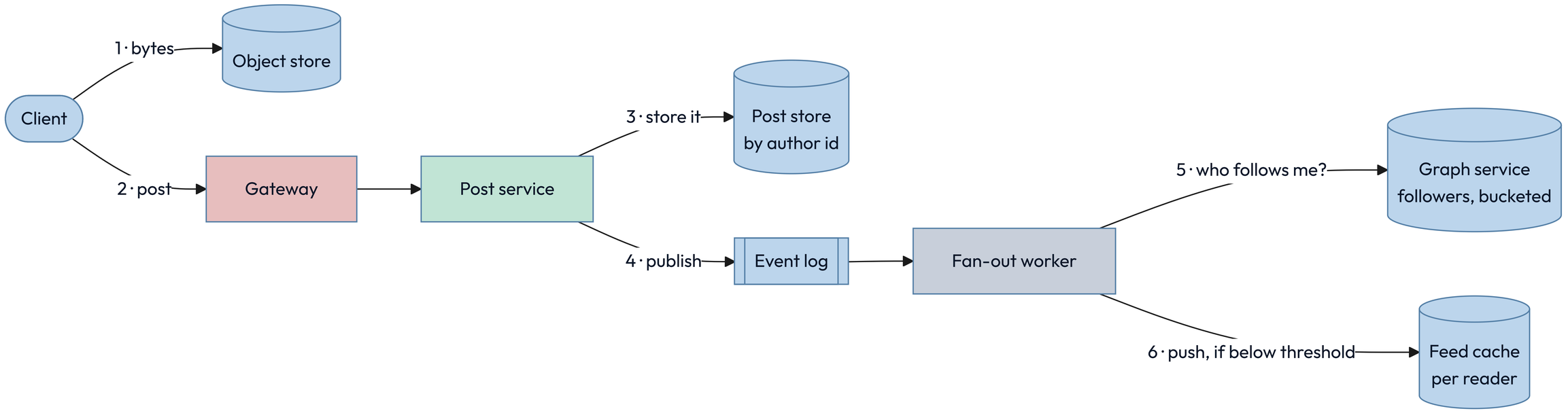
She is a five-hundred-million-edge node that breaks every uniform assumption.

WHAT YOU DO ABOUT IT

You do not resolve the tension. You build a second path and name why it exists.

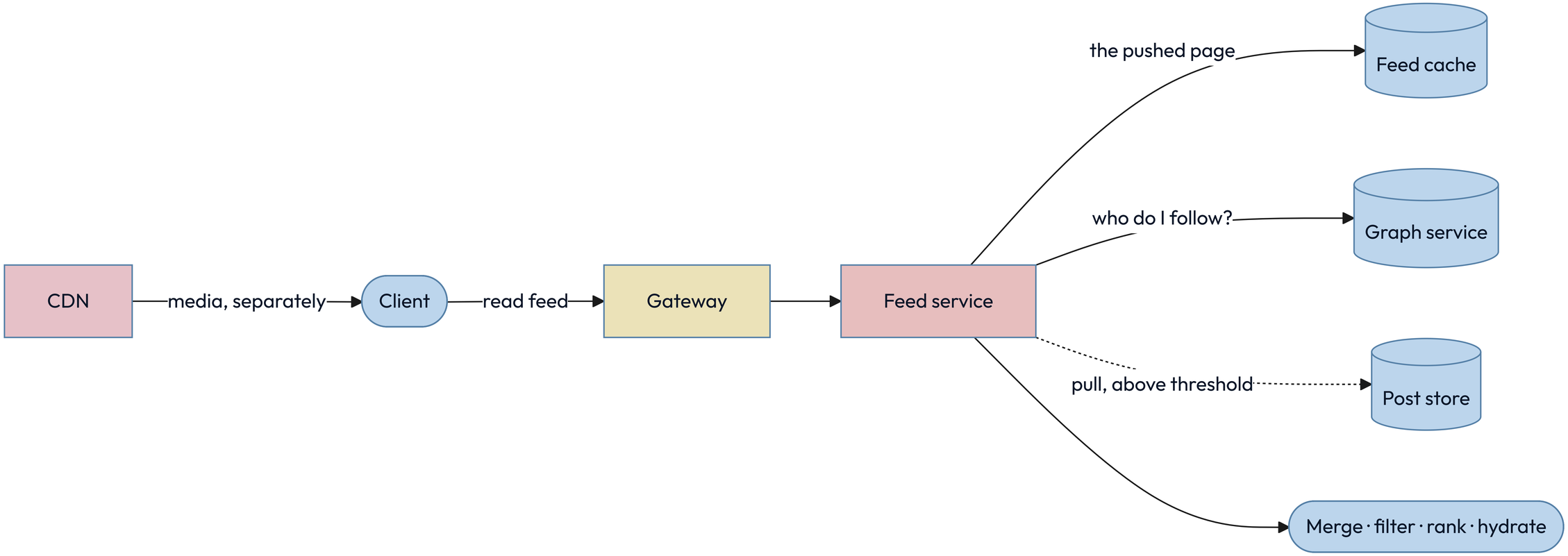
INSTAGRAM · THE WRITE PATH

The graph service is the box everyone forgets to draw.



INSTAGRAM · THE READ PATH

The read path is one lookup, plus whatever the celebrities added.



INSTAGRAM · ASSEMBLING A PAGE

A feed read is four stages, and the last one costs the most.

STEP 01

Fetch and merge

Read the precomputed page, then pull recent posts from the celebrities she follows.

STEP 02

Filter

Drop blocked accounts, deleted posts and private accounts she does not follow.

STEP 03

Rank

Score the candidates. Paginate on a stable key, never on the score.

STEP 04

Hydrate

Fetch each post, its media URLs, its counts, and whether she already liked it.

THE NUMBER WE MISSED

Seventy thousand feed reads a second is really five million lookups a second.

A twenty-item page needs four lookups an item — the post row, the media variants, the counts, and whether this reader liked it. Eighty a page, times seventy thousand pages a second, is 5.6 million. Sizing a fleet from the 70K and meeting the 5.6M in production is the failure the envelope exists to prevent.

Batch every hydration, and cache the post row and its media variants — they barely change. Do not fold the count into the feed entry: that entry is written at post time, when the count is zero, and updating it later is the fan-out this design exists to avoid. Counts get their own keyed store, and "did I like this" is per reader and cannot be folded in at all.

INSTAGRAM · THE LIKELY BUG

Your own post must appear instantly, or people think the upload failed.

The feed is eventually consistent, which is correct for everyone else's posts and completely wrong for your own. A person who posts and does not see it reads that as data loss, not as staleness, and posts again.

THE TELL

Read-your-writes, the consistency rung from part four, finally applied to something.

WRITE YOUR OWN FEED SYNCHRONOUSLY

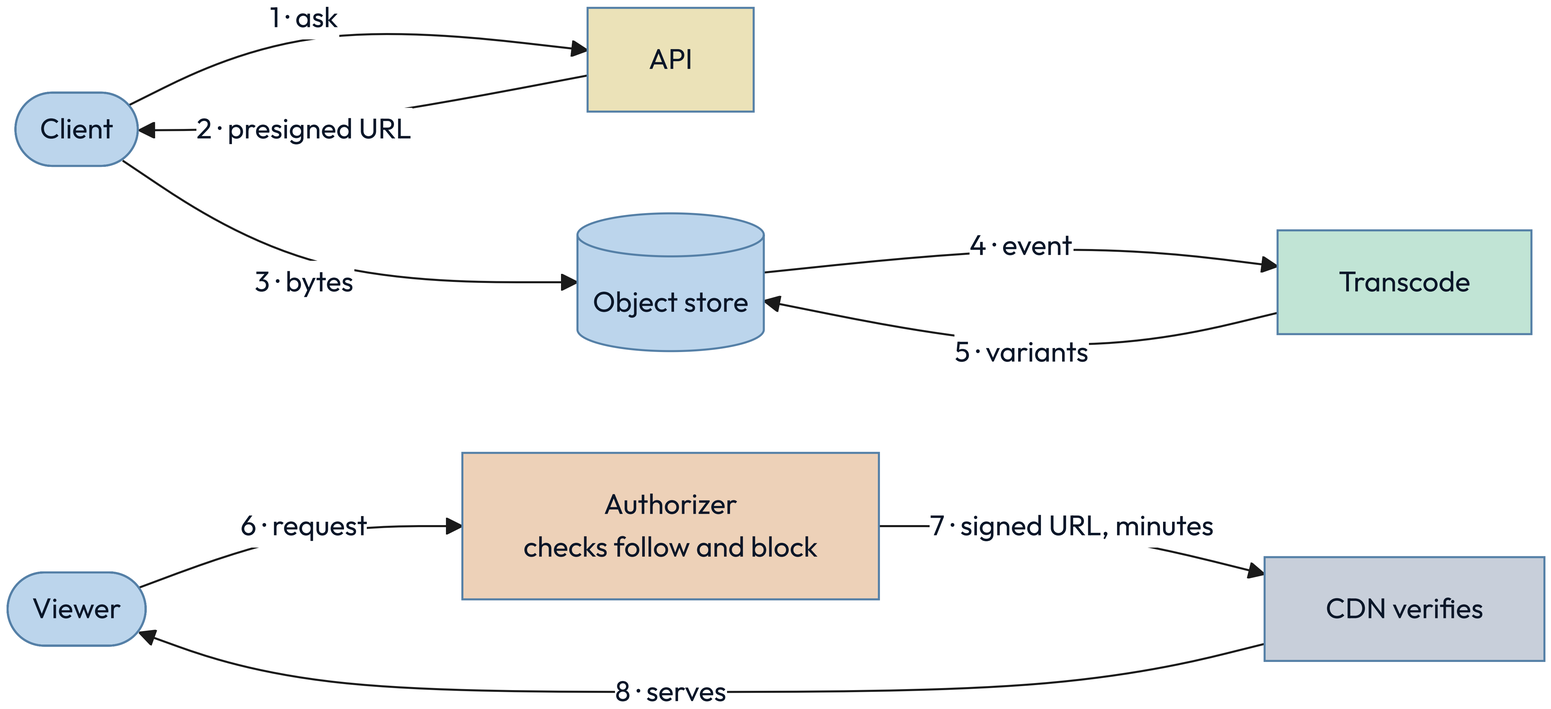
Inside the POST request, before it returns. One extra write, on one key.

AND LET THE CLIENT HELP

It inserts the post it just created optimistically, and reconciles on the next fetch.

INSTAGRAM · THE MEDIA PATH

Media never touches your API servers, and private media never touches an open URL.



INSTAGRAM · WHERE IT BREAKS

Four failures are certain, and each has an answer you already have.

Celebrity post

The fan-out queue floods. Answer: the pull path above the threshold.

Feed cache eviction

Cold readers cost a rebuild. Answer: rebuild lazily from posts and edges.

Redelivery

The queue delivers twice. Answer: the feed entry is keyed on reader and post.

Zone loss

A whole zone disappears. Answer: media replicated, every derived store rebuildable.

INSTAGRAM · THE INVARIANTS

Six sentences hold, or the design is not finished.

A post is visible to every eligible follower who reaches it.	eventually
Media is never lost once an upload is acknowledged.	durable
A like counts exactly once per reader and post.	idempotent
A feed page never repeats or skips an item.	stable cursor
A blocked account is filtered at read, never only at write.	retroactive
Every derived store rebuilds from posts and edges.	rebuildable

06

PART SIX

Every choice came out of a kit, including the one we refused.

THE TIE-BACK · STORE AND RUN

Where each kit entry landed in the Instagram design.

KIT ENTRY	WHERE IT LANDED	WHY THAT ONE
Wide-column	The two edge tables	A partition key plus a sorted range is an adjacency list
Object store	Photo bytes and variants	Large, immutable, fetched by key, kept for years
Key-value	The feed cache	The key is known exactly and nothing is asked of the value
Durable log	Post events into fan-out	Producers outpace consumers, deliberately
Stateless services	Feed and post service	Any instance serves any reader; state lives elsewhere
Bounded queue	Fan-out and transcode	Unbounded, one celebrity means an outage

THE TIE-BACK · SCALE AND DEFENSE

And where the other three kits landed.

KIT ENTRY	WHERE IT LANDED	WHY THAT ONE
Reduce	One cached list per celebrity	Written once, read five hundred million times
Spread	Posts by author, edges by bucket	The bucket keeps a super node off one partition
Defer	Fan-out on write, below the threshold	Moves the cost out of the reader's 200 milliseconds
CDN with versioned URLs	Every photo variant	Same bytes for many readers, and purging is eventual
Bulkhead	Fan-out workers kept off the read path	A burst of ordinary fan-out must not starve the pool serving feeds
Object-level authorization	Every hydration	The endpoint check passes; the object check stops the breach

The most useful entry here is the one we refused.

Every junior asked to design Instagram reaches for a graph database, because the words "social graph" are right there. The data kit already answered it, fifty slides before anybody had heard of a super node.

WHAT THE CARD SAID

Walk away when you have relationships but only ever join two hops.

WHAT THE FEED ACTUALLY DOES

Walks one graph hop, me to the people I follow, then a keyed lookup that is not a traversal.

WHY THE REFUSAL MATTERS MOST

A kit that only says yes is a catalog. One that says no is a tool.

THE ARTIFACT

This is the whole method, and it fits on one page.

```
Protagonist   _____________________  wants to _____________  so that ________
Antagonist    But _________________  —  and it is this big: ____________________

Purpose       ____________________________________________________________
Boundary      In: ________________________  Out: ________________________
Environment   ____________________________________________________________
Constraints   physical ______  economic ______  human ______  legal ______
Invariants    1 ___________________  2 ___________________  3 ____________
Bottleneck    ____________________________  measured at ____________________
Solution type MVP  ·  scaled  ·  optimized  ·  optimal  ·  specialized
```

THE REVIEW

Six questions to ask of any design, starting with your own.

- [ ] Who is the protagonist, and who are we not designing for? — casting
- [ ] What is the antagonist, and what number describes it? — evidence
- [ ] Which rung of the ladder is this, and does everyone agree? — frame
- [ ] What breaks first at ten times the load? — scale
- [ ] What happens when each dependency is slow rather than down? — failure
- [ ] What can one stolen credential reach? — security

WHAT TO DO ON MONDAY

Four moves, and the last one is the one that teaches you.

STEP 01

Pick the unglamorous system

Not a famous one. The service you were debugging on Thursday, or the pipeline nobody wants to own.

STEP 02

Cast it before you draw it

Protagonist and antagonist first, one sentence each. If you cannot name them, you do not understand it yet.

STEP 03

Fill the other seven fields

Let them argue with you. A field you cannot answer is the design question you have been avoiding.

STEP 04

Bring the page to your one-on-one

Ask the person across from you which field you got wrong. That conversation is the whole point of learning this.

HOW TO THINK ABOUT SYSTEMS

Name the person. Name the force. The design follows.

Maya never designed anything on Tuesday, and she used every word in Part one before she went to bed.